



CSC-331

Analysis of Algorithm

Lecture Outline

- Graph
- Types of Graph

Introduction to Graphs

- A graph is a collection of vertices (nodes) connected by edges (links).
- Examples:
 - Cities connected by roads
 - Computers connected in a network
 - People connected by friendships on social media

The Birth of Graph Theory (1736)

- Leonhard Euler — The Founder of Graph Theory
 - Year: 1736
 - Problem: Königsberg Bridge Problem
 - City: Königsberg (now Kaliningrad, Russia)
- Question: Is it possible to cross all seven bridges exactly once?
 - Euler represented land as vertices and bridges as edges, creating the first graph model.

Euler's Solution

- Euler proved the walk was impossible because each vertex had an odd degree.
- His 1736 paper 'Solutio problematis ad geometriam situs pertinentis' laid the foundation of graph theory.

Early Development (18th–19th Century)

- 18th Century:
 - Euler introduced geometry of position (topology)
- 19th Century:
 - Arthur Cayley (1857): Studied trees and molecular structures
 - Gustav Kirchhoff (1847): Used graphs in analyzing electrical circuits

20th Century Expansion

- Graphs became essential tools in various disciplines:
 - Chemistry, biology, and engineering
 - Computer science: data structures, networks, and shortest path algorithms (Dijkstra, 1959)
 - Operations research and social networks

Modern Era (21st Century)

- Applications of Graph Theory:
 - Internet and computer networks
 - Artificial Intelligence and Machine Learning
 - Social Network Analysis
 - Bioinformatics and logistics optimization

Major Contributors

- 1736 - Leonhard Euler: Founded graph theory
- 1847 - Gustav Kirchhoff: Electrical networks
- 1857 - Arthur Cayley: Tree structures
- 1936 - Dénes Kőnig: First graph theory book
- 1959 - Edsger Dijkstra: Shortest path algorithm

Importance of Graphs in AI

- Graphs provide structure for complex relationships.
- They are used in:
 - Knowledge Representation
 - Search Algorithms
 - NLP and Computer Vision
 - Recommendation Systems
 - Graph Neural Networks

Graphs in Knowledge Representation

- Types:
 - Semantic Networks – Concept and relationship model
 - Conceptual Graphs – Natural language-like structures
 - Knowledge Graphs – Large-scale, entity-relation models (e.g., Google)
- Example:
 - (Elon Musk) — (CEO of) — (Tesla)

Graphs in Search Algorithms

- Graphs represent state-space in AI search problems.
- Common Algorithms:
 - BFS: Shortest path (unweighted)
 - DFS: Explore paths
 - A*: Optimal path using heuristics
 - Dijkstra's: Shortest path (weighted)
 - Applications: Game playing, Robotics, Route planning

Graphs in Machine Learning

- Graphs model relational data ignored by traditional ML.
 - Graph-Based Semi-Supervised Learning
 - Nodes = Data points
 - Edges = Similarity links
 - Graph Neural Networks (GNNs)
 - Extend deep learning to graph-structured data

Applications of Graphs in AI/ML

- Knowledge Representation: Knowledge Graphs
- Path Planning: Robotics
- Recommendation Systems: Collaborative filtering
- NLP: Relation extraction
- Bioinformatics: Protein networks
- Vision: Scene graphs
- Cybersecurity: Anomaly detection
- Social Media: Influence analysis

Advantages of Using Graphs

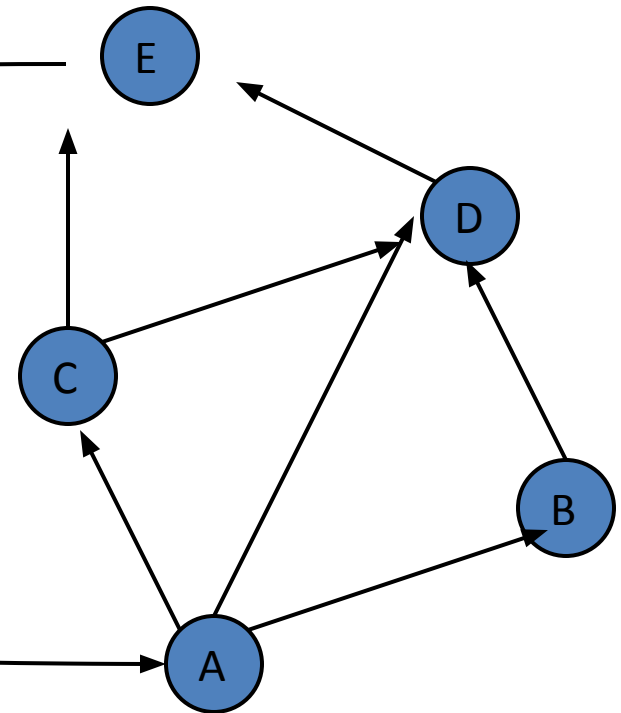
- Represent complex relationships
- Enable reasoning and inference
- Support explainable AI
- Handle non-Euclidean data structures

Limitations of Graphs

- High computational complexity
- Scalability issues in large graphs
- Dynamic changes are hard to track
- Data sparsity affects performance

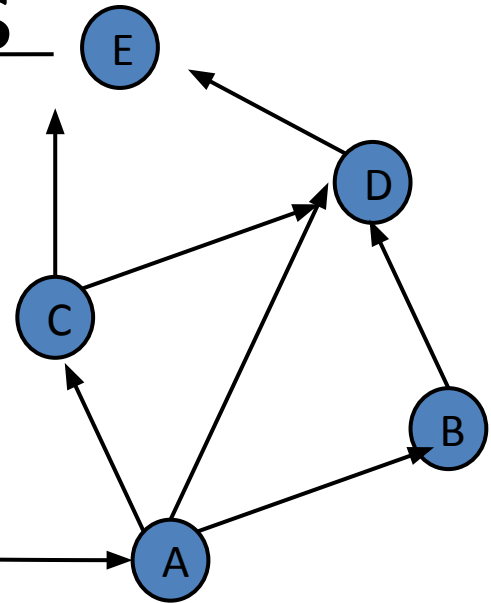
Digraphs

- A **digraph** is a graph whose edges are all directed
 - Short for “directed graph”
- Applications
 - one-way streets
 - flights
 - task scheduling



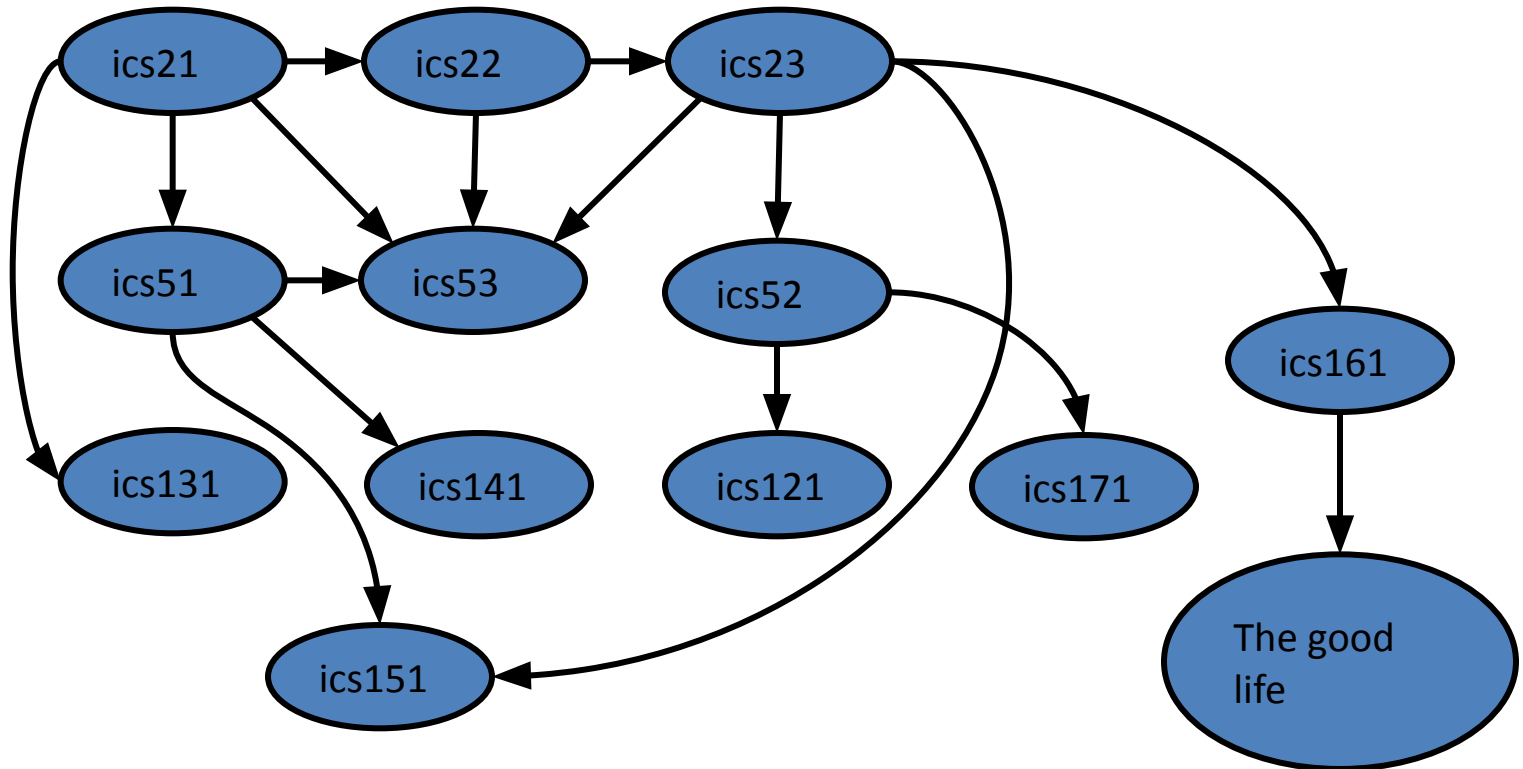
Digraph Properties

- A graph $G=(V,E)$ such that
 - Each edge goes in one direction:
 - Edge (a,b) goes from a to b , but not b to a .
- If G is simple, $m \leq n*(n-1)$.
- If we keep in-edges and out-edges in separate adjacency lists, we can perform listing of in-edges and out-edges in time proportional to their size.



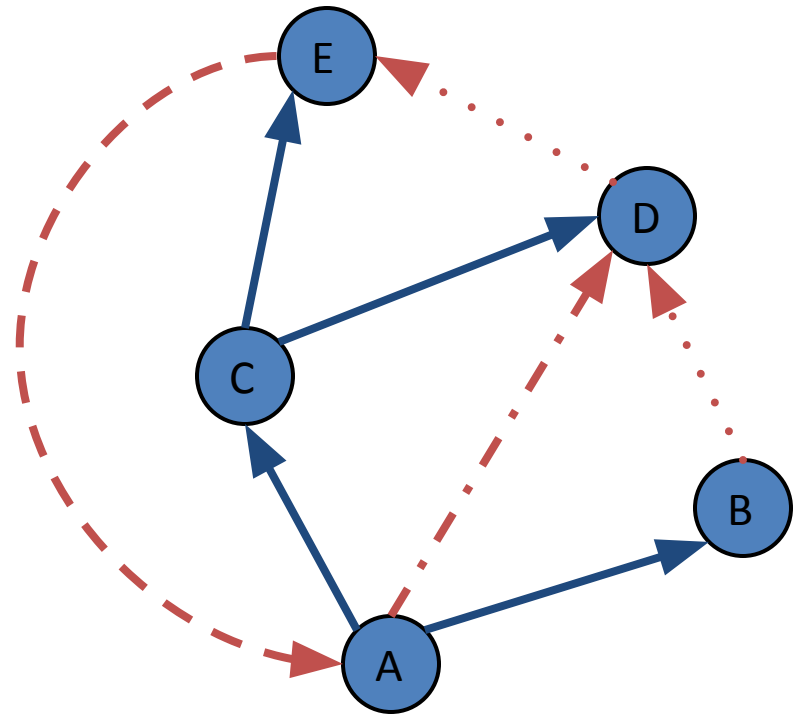
Digraph Application

- **Scheduling:** edge (a,b) means task a must be completed before b can be started



Directed DFS

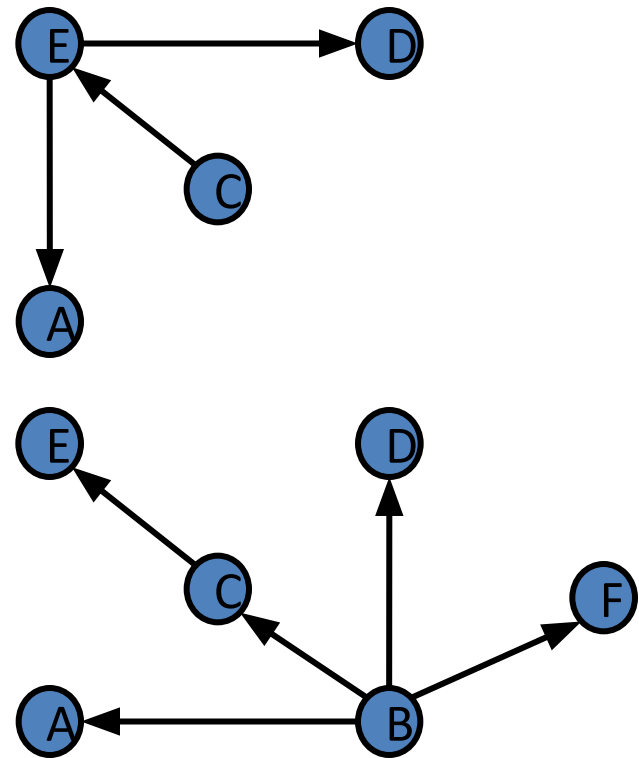
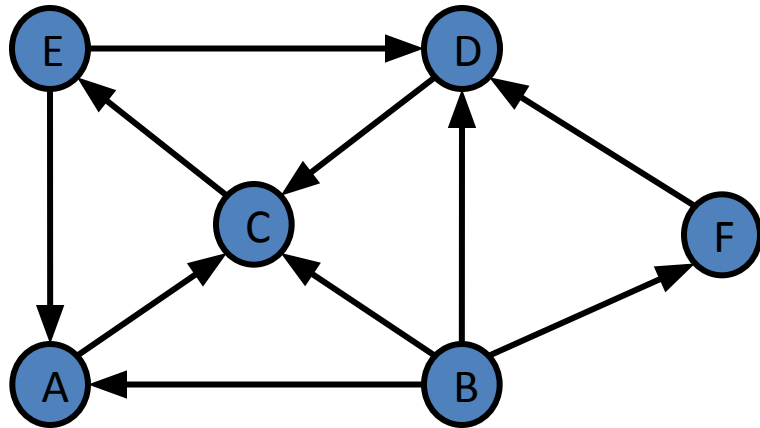
- We can specialize the traversal algorithms (DFS and BFS) to digraphs by traversing edges only along their direction
- In the directed DFS algorithm, we have four types of edges
 - discovery edges
 - back edges
 - forward edges
 - cross edges
- A directed DFS starting at a vertex s determines the vertices reachable from s



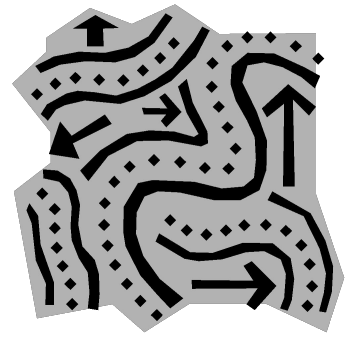
Reachability



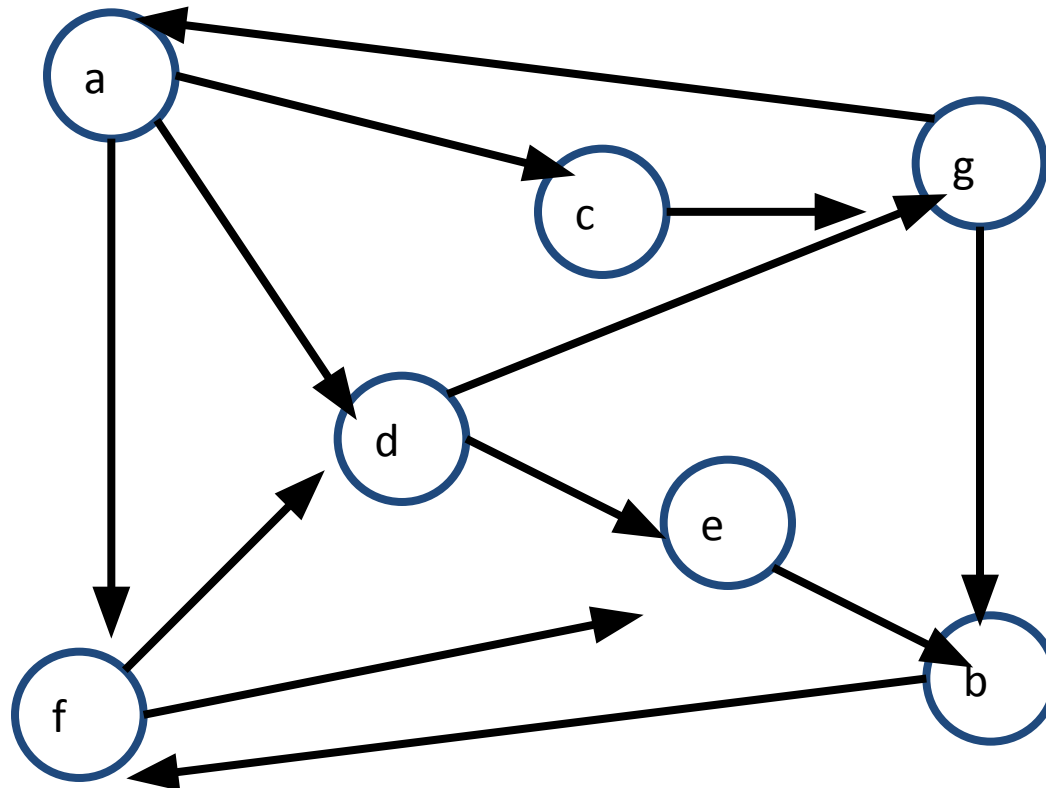
- DFS **tree** rooted at v : vertices reachable from v via directed paths



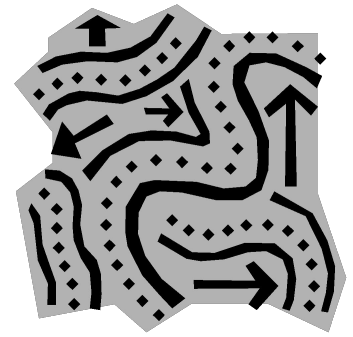
Strong Connectivity



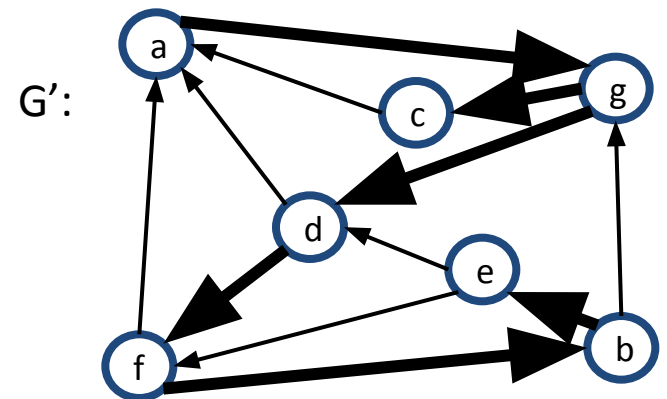
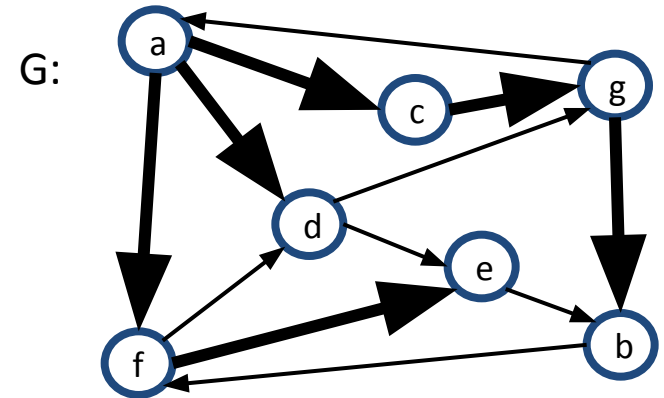
- Each vertex can reach all other vertices



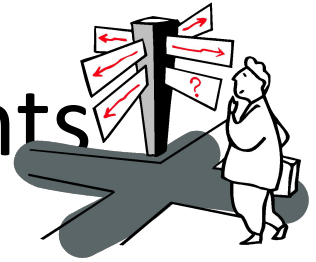
Strong Connectivity Algorithm



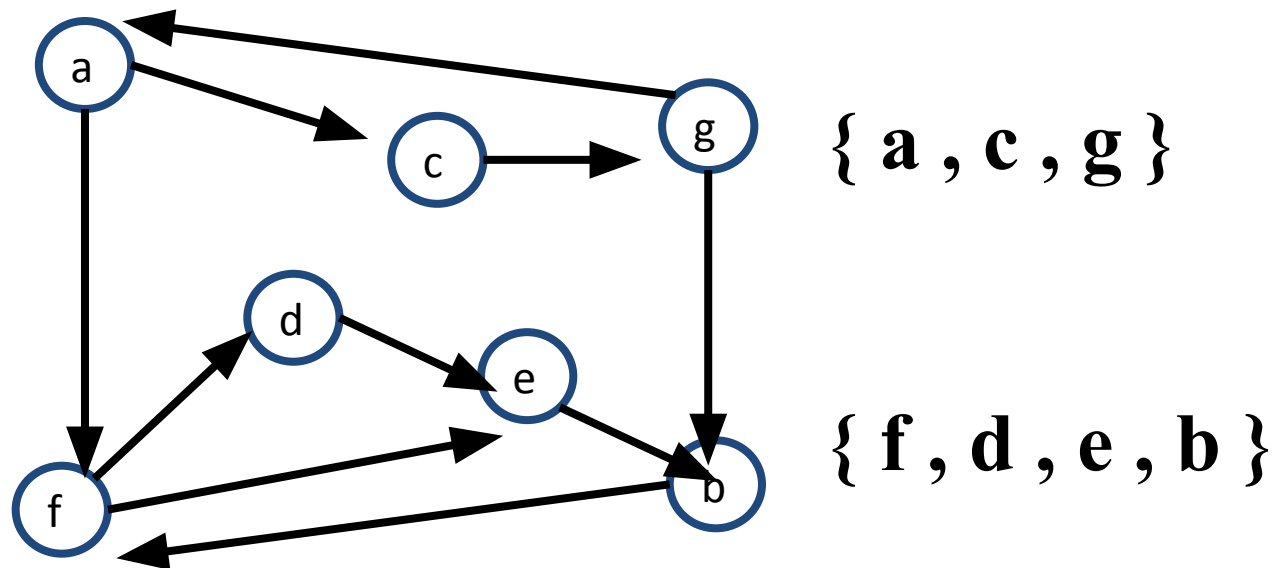
- Pick a vertex v in G .
- Perform a DFS from v in G .
 - If there's a w not visited, print “no”.
- Let G' be G with edges reversed.
- Perform a DFS from v in G' .
 - If there's a w not visited, print “no”.
 - Else, print “yes”.
- Running time: $O(n+m)$.



Strongly Connected Components

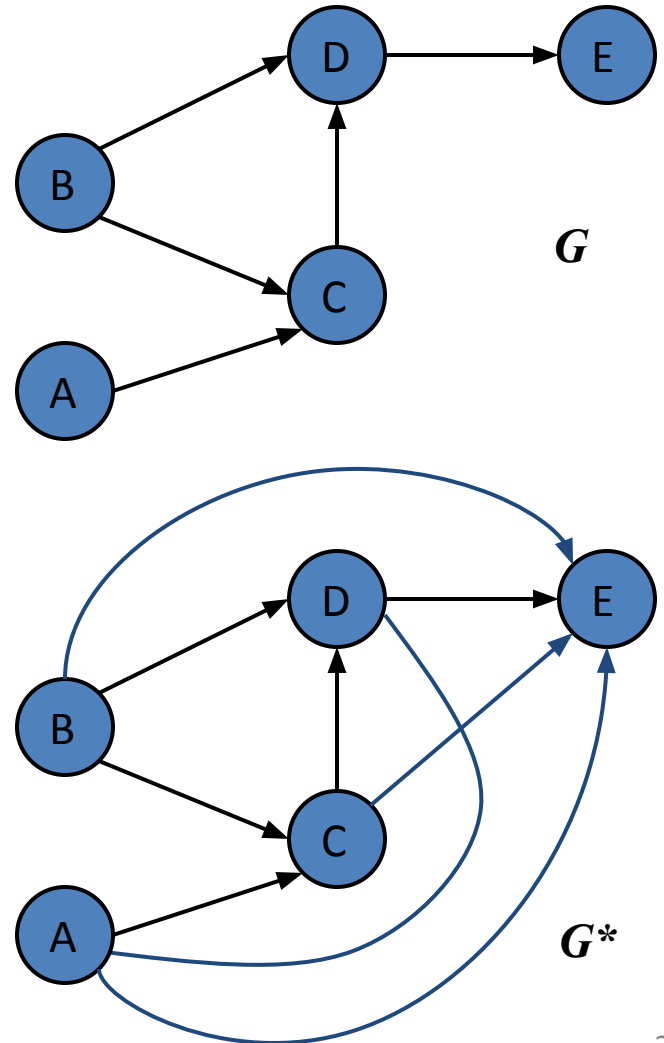


- Maximal subgraphs such that each vertex can reach all other vertices in the subgraph
- Can also be done in $O(n+m)$ time using DFS, but is more complicated (similar to biconnectivity).



Transitive Closure

- Given a digraph G , the transitive closure of G is the digraph G^* such that
 - G^* has the same vertices as G
 - if G has a directed path from u to v ($u \neq v$), G^* has a directed edge from u to v
- The transitive closure provides reachability information about a digraph



Computing the Transitive Closure

- We can perform DFS starting at each vertex
 - $O(n(n+m))$



If there's a way to get from **A** to **B** and from **B** to **C**, then there's a way to get from **A** to **C**.

- Alternatively ... Use dynamic programming: The Floyd-Warshall Algorithm

Minimum Spanning Tree(MST)

Future Trends

- Graph Transformers for relational reasoning
- Dynamic Graph Learning for evolving data
- Graph Federated Learning for privacy
- Integration with LLMs for structured reasoning

